Question-1

Khoi Huynh

2026-10-26

What do you imagine is the reason for log-transforming the protein levels in biomarker-raw.csv? (Hint: look at the distribution of the raw values for a sample of proteins.)

Setup

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.1.5
## v forcats   1.0.1     v stringr   1.5.2
## v ggplot2   4.0.3     v tibble    3.3.0
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at /Users/shenrunxin/R/Pstat197/197a-module1
```

```
#getting protein names
var_names <- read_csv(here("data", "biomarker-raw.csv"),
                      col_names = F,
                      n_max = 2,
                      col_select = -(1:2)) |>
  t() |>
  as_tibble() |>
  rename(name = V1,
         abbreviation = V2) |>
  na.omit()
```

```
## Rows: 2 Columns: 1318
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1318): X3, X4, X5, X6, X7, X8, X9, X10, X11, X12, X13, X14, X15, X16, X...
```

```
##
## i Use ‘spec()‘ to retrieve the full column specification for this data.
## i Specify the column types or set ‘show_col_types = FALSE‘ to quiet this message.
```

```
## Warning: The ‘x‘ argument of ‘as_tibble.matrix()‘ must have unique column names if
## ‘.name_repair‘ is omitted as of tibble 2.0.0.
## i Using compatibility ‘.name_repair‘.
## This warning is displayed once per session.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```

```
#load in data
biomarker_raw <- read_csv(here("data", "biomarker-raw.csv"),
                          skip = 2,
                          col_select = -2L,
                          col_names = c("group",
                                        "empty",
                                        pull(var_names, abbreviation),
                                        "ados"),
                          na = c("-", "")) |>
  filter(!is.na(group)) |>
  select(group, ados, everything())
```

```
## Rows: 155 Columns: 1319
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr    (1): group
## dbl (1318): CHIP, CEBPB, NSE, PIAS4, IL-10 Ra, STAT3, IRF1, c-Jun, Mcl-1, OA...
##
## i Use ‘spec()‘ to retrieve the full column specification for this data.
## i Specify the column types or set ‘show_col_types = FALSE‘ to quiet this message.
```

Distributions of Raw Values

```
#picking 4 random proteins
set.seed(2026)

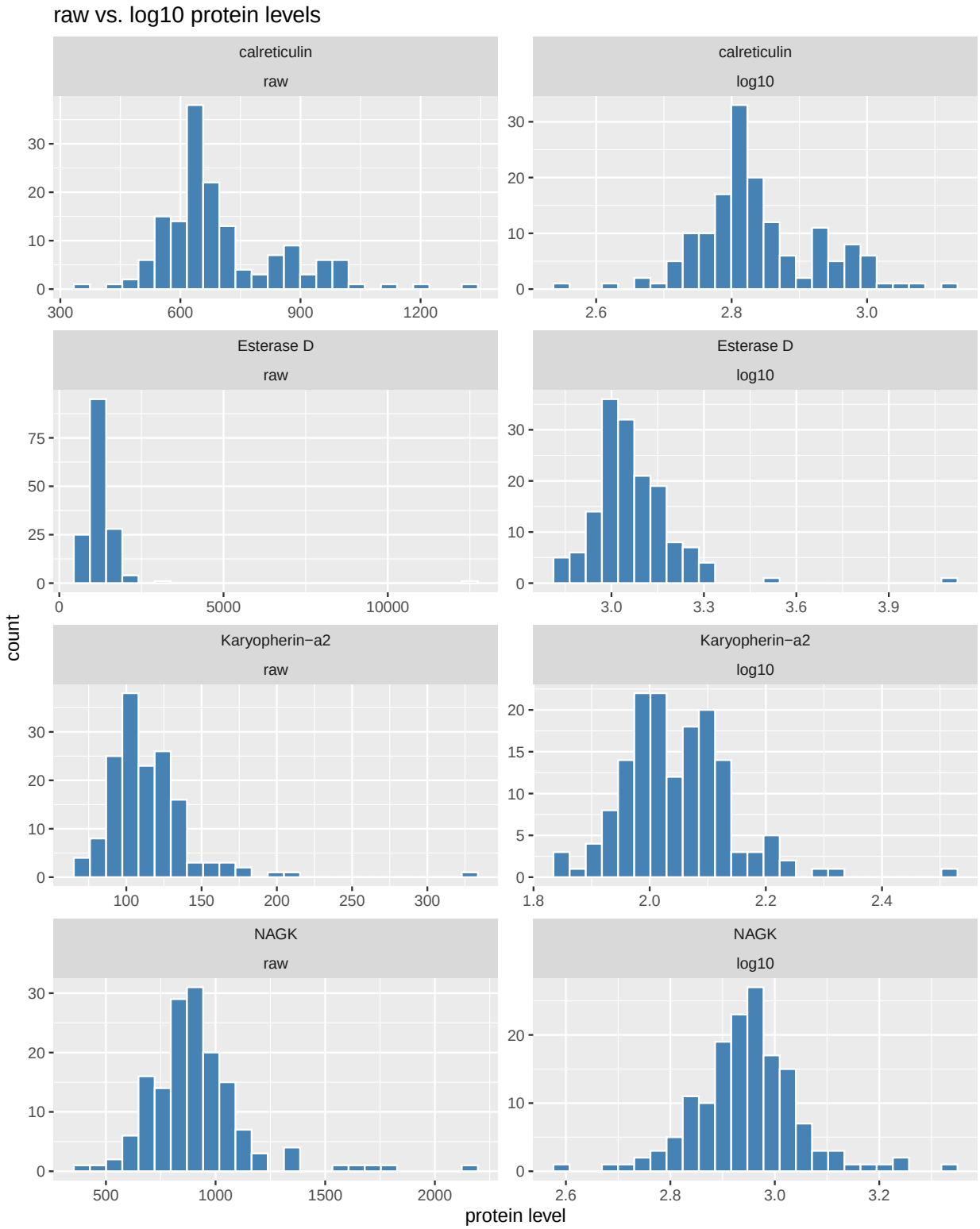
protein_cols <- setdiff(names(biomarker_raw), c("group","ados"))
chosen <- sample(protein_cols, 4)

hist_data <- biomarker_raw |>
  select(all_of(chosen)) |>
  pivot_longer(everything(), names_to = "protein", values_to = "raw") |>
  mutate(log10 = log10(raw)) |>
  pivot_longer(c(raw, log10), names_to = "scale", values_to = "value") |>
  mutate(scale = factor(scale, levels = c("raw", "log10")))

hist_plot <- ggplot(hist_data, aes(x = value)) +
  geom_histogram(bins = 25, fill = "steelblue", color = "white") +
  facet_wrap(vars(protein, scale), scales = "free", ncol = 2) +
```

```
  labs(x = "protein level", y = "count",
       title = "raw vs. log10 protein levels")

hist_plot
```




```
ggsave(here("results", "figures", "q1-raw-vs-log-histograms.png"),
       plot = hist_plot, width = 8, height = 10, dpi = 300)
```

Why log-transform the protein levels? On the raw scale, the protein levels are generally right skewed.

For a typical protein, most subjects' values cluster at the low end of its range, with a long tail of a few much larger values, and some proteins having extreme readings that are above the rest by a lot. Because of this, the mean of a protein is typically pulled above its median.

After applying the log_{10} transformation, the distributions become much closer to symmetric and roughly bell shaped. Regardless some proteins still end up slightly left skewed, since the the log compresses the larger values more than the smaller ones, but the heavy right tails and extreme values are no longer dominating. In general, log transforming compresses the scale of the data, making it more symmetric and easier to read. With this data set in particular, the protein values vary widely, so applying the log transformation makes the distribution easier to visualize and brings them closer to normality.

Skewness Summary

```
skew <- function(x) {
  x <- x[!is.na(x)]
  mean((x - mean(x))^3) / sd(x)^3
}

skew_summary <- tibble(
  protein = protein_cols,
  raw = map_dbl(biomarker_raw[protein_cols], skew),
  log10 = map_dbl(biomarker_raw[protein_cols], ~skew(log10(.x))),
  mean_gt_median = map_lgl(biomarker_raw[protein_cols],
                           ~ mean(.x, na.rm = TRUE) > median(.x, na.rm = TRUE)))

skew_table <- skew_summary |>
  summarise(median_skew_raw = median(raw),
            median_skew_log10 = median(log10),
            pct_right_skewed_raw = mean(raw > 0) * 100,
            pct_mean_above_median = mean(mean_gt_median) * 100)

skew_table
```

```
## # A tibble: 1 x 4
##   median_skew_raw median_skew_log10 pct_right_skewed_raw pct_mean_above_median
##             <dbl>             <dbl>                <dbl>                 <dbl>
## 1            2.26             0.831                 96.7                  96.9
```

```
write_csv(skew_table, here("results", "tables", "q1-skewness-summary.csv"))
```

Across all proteins, about 97% are right skewed on the raw scale, with a middle skewness of 2.26 and the mean above the median for nearly all of them. After doing the log_{10} transformation, the median skewness drops to 0.83, which is still a little right skewed but a large reduction, and it confirms what we said earlier that the log brings the distributions much closer to symmetric.